

Business Logic Documentation

Network Monitoring System

Document Version: 1.0.0

Table of Contents

1. [System Overview](#)
2. [Architectural Patterns](#)
3. [Core Business Rules](#)
4. [Data Flow Architecture](#)
5. [Component Interactions](#)
6. [Protocol Processing Logic](#)
7. [Statistics & Analytics Business Logic](#)
8. [Storage & Persistence Logic](#)
9. [User Interface Business Logic](#)
10. [Error Handling & Edge Cases](#)
11. [Configuration Management](#)
12. [State Management](#)

System Overview

The Network Monitoring System is a comprehensive network traffic analysis tool that captures, parses, stores, and analyzes network packets in real-time. The system follows a **multi-threaded producer-consumer architecture** with separation of concerns across capture, processing, analysis, and storage layers.

Key Business Objectives

- **Real-time Packet Capture:** Capture packets from network interfaces using libpcap

- **Protocol Analysis:** Parse and identify network protocols (Ethernet, IP, TCP, UDP, ICMP, HTTP, HTTPS, DNS, DHCP, ARP)
- **Statistical Analysis:** Calculate real-time statistics including bandwidth, protocol distribution, host activity, and connection tracking
- **Data Persistence:** Store packet data in SQLite database for historical analysis
- **Dual Interface Support:** Provide both GUI (Qt-based) and CLI interfaces
- **Filtering Capabilities:** Support BPF (Berkeley Packet Filter) expressions for packet filtering

Technology Stack

- **Framework:** Qt6 (for GUI), C++20 Standard
- **Packet Capture:** libpcap
- **Database:** SQLite3
- **Threading:** std::thread with mutex/condition_variable synchronization
- **Pattern:** Producer-Consumer, Observer, Singleton

Architectural Patterns

1. Producer-Consumer Pattern

Location: src/core/NetworkMonitor.cpp

The system implements a **multi-stage producer-consumer pipeline**:

```
NetworkMonitor::NetworkMonitor()
    : pcap_handle_(nullptr)
    , running_(false)
    , statistics_(std::make_unique<Statistics>())
    , data_store_(std::make_unique<DataStore>()) {
}
```

```
NetworkMonitor::~~NetworkMonitor() {
    stop();
    if (pcap_handle_) {
        pcap_close(pcap_handle_);
    }
}
```

```

}

void NetworkMonitor::start() {
    if (running_) {
        return;
    }

    char errbuf[PCAP_ERRBUF_SIZE];
    pcap_handle_ = pcap_open_live(interface_.c_str(), BUFSIZ, 1, 1000, errbuf);
    if (!pcap_handle_) {
        throw std::runtime_error("Failed to open interface: " + std::string(errbuf));
    }

    if (!filter_.empty()) {
        struct bpf_program fp;
        if (pcap_compile(pcap_handle_, &fp, filter_.c_str(), 0, PCAP_NETMASK_UNKNOWN) == -1)
        {
            throw std::runtime_error("Failed to compile filter: " +
std::string(pcap_geterr(pcap_handle_)));
        }
        if (pcap_setfilter(pcap_handle_, &fp) == -1) {
            pcap_freecode(&fp);
            throw std::runtime_error("Failed to set filter: " +
std::string(pcap_geterr(pcap_handle_)));
        }
        pcap_freecode(&fp);
    }

    running_ = true;
    capture_thread_ = std::thread(&NetworkMonitor::captureThread, this);
    process_thread_ = std::thread(&NetworkMonitor::processThread, this);
    analyze_thread_ = std::thread(&NetworkMonitor::analyzeThread, this);
    store_thread_ = std::thread(&NetworkMonitor::storeThread, this);
}

```

Pipeline Stages:

1. **Capture Thread** (Producer) - Reads packets from network interface
2. **Process Thread** (Consumer/Producer) - Processes packets and triggers callbacks
3. **Analyze Thread** (Consumer) - Updates statistics in real-time
4. **Store Thread** (Consumer) - Persists packets to database

2. Singleton Pattern

Location: src/config/ConfigManager.cpp

Configuration management uses Singleton pattern:

```
ConfigManager& ConfigManager::getInstance() {  
    static ConfigManager instance;  
    return instance;  
}
```

3. Observer Pattern

Location: src/core/NetworkMonitor.cpp

Packet callbacks enable observer pattern for real-time UI updates:

```
void NetworkMonitor::addPacketCallback(std::function<void(const Packet&> callback) {  
    packet_callbacks_.push_back(std::move(callback));  
}
```

4. MVC-like Pattern (GUI Layer)

Location: src/gui/MainWindow.cpp

The GUI follows a Model-View-Controller-like pattern:

- **Model:** NetworkMonitor, Statistics, DataStore
- **View:** Qt Widgets (MainWindow, StatisticsWidget, PacketsWidget, etc.)
- **Controller:** MainWindow coordinates between model and view

Core Business Rules

BR-001: Packet Capture Rules

File: src/core/NetworkMonitor.cpp:88-105

Rule: Only capture packets when running_ is true and interface is valid.

```
void NetworkMonitor::captureThread() {
    while (running_) {
        struct pcap_pkthdr* header;
        const u_char* data;
        int result = pcap_next_ex(pcap_handle_, &header, &data);

        if (result == 1) {
            Packet packet(data, header->len, header->ts);
            {
                std::lock_guard<std::mutex> lock(queue_mutex_);
                packet_queue_.push(std::move(packet));
            }
            queue_cv_.notify_one();
        } else if (result == -1) {
            Logger::error("Error reading packet: " + std::string(pcap_geterr(pcap_handle_)));
        }
    }
}
```

Business Logic:

- If pcap_next_ex returns 1: Packet captured successfully → enqueue
- If pcap_next_ex returns -1: Error occurred → log error, continue monitoring
- If pcap_next_ex returns 0: Timeout (normal) → continue loop

BR-002: Filter Compilation Rules

File: src/core/NetworkMonitor.cpp:31-41

Rule: BPF filters must be compiled and set before packet capture starts.

Business Logic:

1. If filter string is empty → No filter applied
2. If filter compilation fails → Throw exception, prevent start
3. If filter setting fails → Throw exception, prevent start
4. Filter must be freed after setting to prevent memory leaks

BR-003: Thread Safety Rules

File: Multiple locations using mutexes

Rule: All shared data structures must be protected by mutexes.

Shared Resources:

- packet_queue_ - Protected by queue_mutex_
- statistics_ - Protected by internal mutex in Statistics class
- data_store_ - Protected by queue mutex and database transactions

BR-004: Packet Queue Management

File: src/core/NetworkMonitor.cpp:107-124

Rule: Multiple consumer threads read from the same queue (no packet duplication).

```
void NetworkMonitor::processThread() {
    while (running_) {
        std::unique_lock<std::mutex> lock(queue_mutex_);
        queue_cv_.wait(lock, [this] { return !running_ || !packet_queue_.empty(); });

        if (!running_ && packet_queue_.empty()) {
            break;
        }

        Packet packet = std::move(packet_queue_.front());
        packet_queue_.pop();
        lock.unlock();

        for (const auto& callback : packet_callbacks_) {
            callback(packet);
        }
    }
}
```

```

    }
}
}

```

Critical Business Logic:

- Each consumer thread makes a **copy** of the packet (not move)
- This allows parallel processing: one packet can be processed, analyzed, and stored simultaneously
- Queue is shared but synchronization ensures thread safety

BR-005: Protocol Identification Rules

File: src/protocols/Packet.cpp:12-31

Rule: Protocol identification follows a hierarchical parsing strategy.

Parsing Hierarchy:

1. **Ethernet Layer** → Determine IP version or ARP
2. **IP Layer** (IPv4/IPv6) → Extract source/dest addresses, determine transport protocol
3. **Transport Layer** (TCP/UDP/ICMP) → Extract ports, sequence numbers
4. **Application Layer** → Identify by port numbers (HTTP:80, HTTPS:443, DNS:53, DHCP:67/68)

```

void Packet::determineApplicationProtocol() {
    if (isTCP()) {
        if (source_port == 80 || destination_port == 80) {
            protocol = Protocol::HTTP;
        } else if (source_port == 443 || destination_port == 443) {
            protocol = Protocol::HTTPS;
        }
    }
    } else if (isUDP()) {
        if (source_port == 53 || destination_port == 53) {
            protocol = Protocol::DNS;
        } else if (source_port == 67 || source_port == 68 ||
            destination_port == 67 || destination_port == 68) {
            protocol = Protocol::DHCP;
        }
    }
}

```

```
}  
}
```

BR-006: Malformed Packet Detection

File: src/protocols/Packet.cpp:34-37, 62-65

Rule: Packets are marked as malformed if header parsing fails due to insufficient length.

```
void Packet::parseEthernet() {  
    if (length < sizeof(struct ether_header)) {  
        is_malformed = true;  
        return;  
    }  
}
```

Business Logic:

- Each protocol parser checks minimum header size
- If packet length < required header size → is_malformed = true
- Malformed packets are still processed but flagged for error statistics

BR-007: Connection Identification Rules

File: src/analysis/Statistics.cpp:157-167

Rule: Connection IDs are generated in a canonical format to ensure bidirectional connections share the same ID.

```
std::string Statistics::generateConnectionId(const Packet& packet) const {  
    std::stringstream ss;  
    if (packet.source_address < packet.destination_address) {  
        ss << packet.source_address << ":" << packet.source_port << "-"  
            << packet.destination_address << ":" << packet.destination_port;  
    } else {  
        ss << packet.destination_address << ":" << packet.destination_port << "-"  
            << packet.source_address << ":" << packet.source_port;  
    }  
    return ss.str();  
}
```

```
}
```

Business Logic:

- Connection ID format: lower_ip:port-higher_ip:port
- Ensures $A \rightarrow B$ and $B \rightarrow A$ packets map to the same connection
- Only TCP and UDP packets generate connection stats

BR-008: Connection Timeout Rules

File: src/analysis/Statistics.cpp:145-155

Rule: Connections are considered inactive after 5 minutes of no activity.

```
void Statistics::cleanupInactiveConnections() {
    auto now = std::chrono::system_clock::now();
    for (auto it = connection_stats_.begin(); it != connection_stats_.end(); ) {
        if (now - it->second.last_seen > CONNECTION_TIMEOUT) {
            it->second.is_active = false;
            it = connection_stats_.erase(it);
        } else {
            ++it;
        }
    }
}
```

Business Logic:

- Constant: CONNECTION_TIMEOUT = 300 seconds (5 minutes)
- Inactive connections are removed from memory to prevent unbounded growth
- Cleanup runs after each packet statistics update

BR-009: Bandwidth Calculation Rules

File: src/analysis/Statistics.cpp:111-137

Rule: Bandwidth is calculated per second and stored in history.

```

void Statistics::updateBandwidthStats(const Packet& packet) {
    auto now = std::chrono::system_clock::now();
    auto elapsed = std::chrono::duration_cast<std::chrono::seconds>(now -
last_bandwidth_update_).count();

    if (elapsed >= 1) {
        // Update bandwidth history
        bandwidth_history_.push_back({now, current_bandwidth_});

        // Keep only the last hour of history
        while (bandwidth_history_.size() > MAX_BANDWIDTH_HISTORY) {
            bandwidth_history_.erase(bandwidth_history_.begin());
        }

        // Calculate average bandwidth
        double sum = 0.0;
        for (const auto& entry : bandwidth_history_) {
            sum += entry.second;
        }
        average_bandwidth_ = sum / bandwidth_history_.size();

        // Reset current bandwidth
        current_bandwidth_ = 0.0;
        last_bandwidth_update_ = now;
    }

    current_bandwidth_ += packet.length * 8.0; // Convert bytes to bits
}

```

Business Logic:

- Bandwidth measured in **bits per second** (bytes × 8)
- History updated every 1 second
- Maximum history: 3600 entries (1 hour at 1-second intervals)
- Average bandwidth = sum of all historical values / count

BR-010: Database Batch Insert Rules

File: src/storage/DataStore.cpp:90-103, 153-170

Rule: Packets are batched and inserted in transactions for performance.

```
void DataStore::storeThread() {
    while (running_) {
        std::unique_lock<std::mutex> lock(queue_mutex_);
        queue_cv_.wait_for(lock, FLUSH_INTERVAL, [this] {
            return !running_ || packet_queue_.size() >= BATCH_SIZE;
        });

        if (!running_ && packet_queue_.empty()) {
            break;
        }

        batchInsert();
    }
}
```

Business Logic:

- Batch size: 1000 packets
- Flush interval: 5 seconds
- Transactions: BEGIN → INSERT loop → COMMIT (or ROLLBACK on error)

BR-011: Statistics Update Rules

File: src/analysis/Statistics.cpp:10-23

Rule: Each packet triggers updates across multiple statistical dimensions atomically.

```
void Statistics::update(const Packet& packet) {
    std::lock_guard<std::mutex> lock(mutex_);

    total_packets_++;
    total_bytes_ += packet.length;
```

```

    updateProtocolStats(packet);
    updateHostStats(packet);
    updateConnectionStats(packet);
    updateBandwidthStats(packet);
    updateErrorStats(packet);

    cleanupInactiveConnections();
}

```

Business Logic:

- Single mutex lock for entire update (atomic operation)
- Updates: total counts, protocol stats, host stats, connection stats, bandwidth, errors
- Cleanup runs after updates

BR-012: Retransmission Detection Rules

File: src/analysis/Statistics.cpp:99-108

Rule: TCP retransmissions detected by comparing sequence numbers.

```

// Detect retransmissions for TCP
if (packet.isTCP()) {
    // Simple retransmission detection based on sequence numbers
    // This is a basic implementation and might need improvement
    static std::unordered_map<std::string, uint32_t> last_seq;
    if (last_seq.count(connection_id) && packet.sequence_number ==
last_seq[connection_id]) {
        stats.retransmission_count++;
    }
    last_seq[connection_id] = packet.sequence_number;
}

```

Business Logic:

- Basic heuristic: If sequence number matches previous for same connection → retransmission

- Note: Comment indicates this is simplified and may need improvement
- Only tracks retransmissions for TCP packets

BR-013: Application Initialization Rules

File: src/main.cpp:37-68

Rule: Application must initialize in specific order with error handling.

```
try {
    // Initialize logger
    std::string log_file = parser.value(logFileOption).toStdString();
    if (log_file.empty()) {
        log_file = "network_monitor.log";
    }

    std::string log_level = parser.value(logLevelOption).toStdString();
    Logger::Level level = Logger::Level::INFO;
    if (log_level == "debug") level = Logger::Level::DEBUG;
    else if (log_level == "warning") level = Logger::Level::WARNING;
    else if (log_level == "error") level = Logger::Level::ERROR;
    else if (log_level == "fatal") level = Logger::Level::FATAL;

    Logger::init(log_file, level);
    Logger::info("Application started");

    // Create network monitor
    auto monitor = std::make_unique<NetworkMonitor>();

    // Set interface if specified
    std::string interface = parser.value(interfaceOption).toStdString();
    if (!interface.empty()) {
        monitor->setInterface(interface);
    }

    // Set filter if specified
    std::string filter = parser.value(filterOption).toStdString();
    if (!filter.empty()) {
```

```

        monitor->setFilter(filter);
    }

    // Create and show main window
    MainWindow main_window(monitor.get());
    main_window.show();

```

Business Logic:

- Initialization order: Logger → NetworkMonitor → MainWindow
- Default log file: "network_monitor.log"
- Default log level: INFO
- Interface and filter are optional (can be set via GUI later)

BR-014: GUI Update Rules

File: src/gui/MainWindow.cpp:180-206

Rule: GUI updates occur at fixed intervals when monitoring is active.

```

void MainWindow::updateDisplay() {
    if (!is_monitoring_) {
        return;
    }

    // Update current tab
    switch (tab_widget_->currentIndex()) {
        case 0: // Statistics
            statistics_widget_->update();
            break;
        case 1: // Connections
            connections_widget_->update();
            break;
        case 2: // Packets
            packets_widget_->update();
            break;
        case 3: // Bandwidth
            bandwidth_widget_->update();

```

```

        break;
    }

    // Update status bar
    auto stats = monitor_ -> getStatistics();
    statusBar() -> showMessage(QString("Packets: %1 | Bandwidth: %2 bps")
        .arg(stats.getTotalPackets())
        .arg(stats.getCurrentBandwidth()));
}

```

Business Logic:

- Update interval: 1000ms (1 second) - configured in toggleMonitoring
- Only updates if `is_monitoring_` is true
- Updates only the active tab widget
- Status bar always shows current packet count and bandwidth

BR-015: Logging Level Rules

File: `src/utils/Logger.cpp:51-74`

Rule: Logging respects level hierarchy and outputs to file and console.

```

void Logger::log(Level level, const std::string& message) {
    if (!initialized_) {
        std::cerr << "Logger not initialized" << std::endl;
        return;
    }

    if (level < current_level_) {
        return;
    }

    std::lock_guard<std::mutex> lock(mutex_);
    std::string timestamp = getTimestamp();
    std::string level_str = levelToString(level);

    std::string log_message = timestamp + " [" + level_str + "] " + message + "\n";
}

```

```
*log_file_ << log_message;
log_file_->flush();

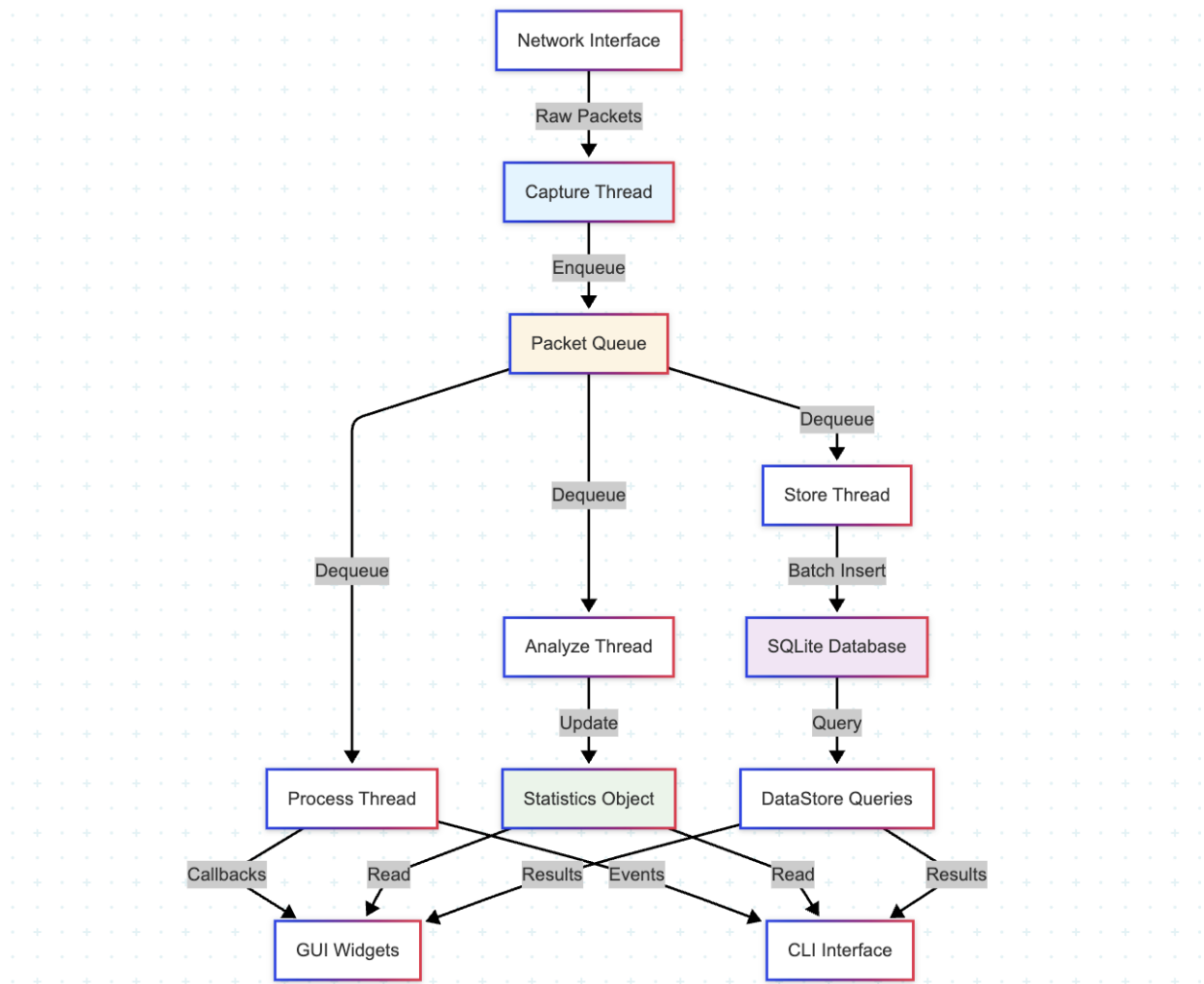
// Also output to console for ERROR and FATAL levels
if (level >= Level::ERROR) {
    std::cerr << log_message;
}
}
```

Business Logic:

- Level hierarchy: DEBUG < INFO < WARNING < ERROR < FATAL
- Messages below current level are filtered
- All messages written to log file
- ERROR and FATAL also output to stderr
- Thread-safe logging with mutex

Data Flow Architecture

High-Level Data Flow



flowchart TD

A[Network Interface] -->|Raw Packets| B[Capture Thread]

B -->|Enqueue| C[Packet Queue]

C -->|Dequeue| D[Process Thread]

C -->|Dequeue| E[Analyze Thread]

C -->|Dequeue| F[Store Thread]

D -->|Callbacks| G[GUI Widgets]

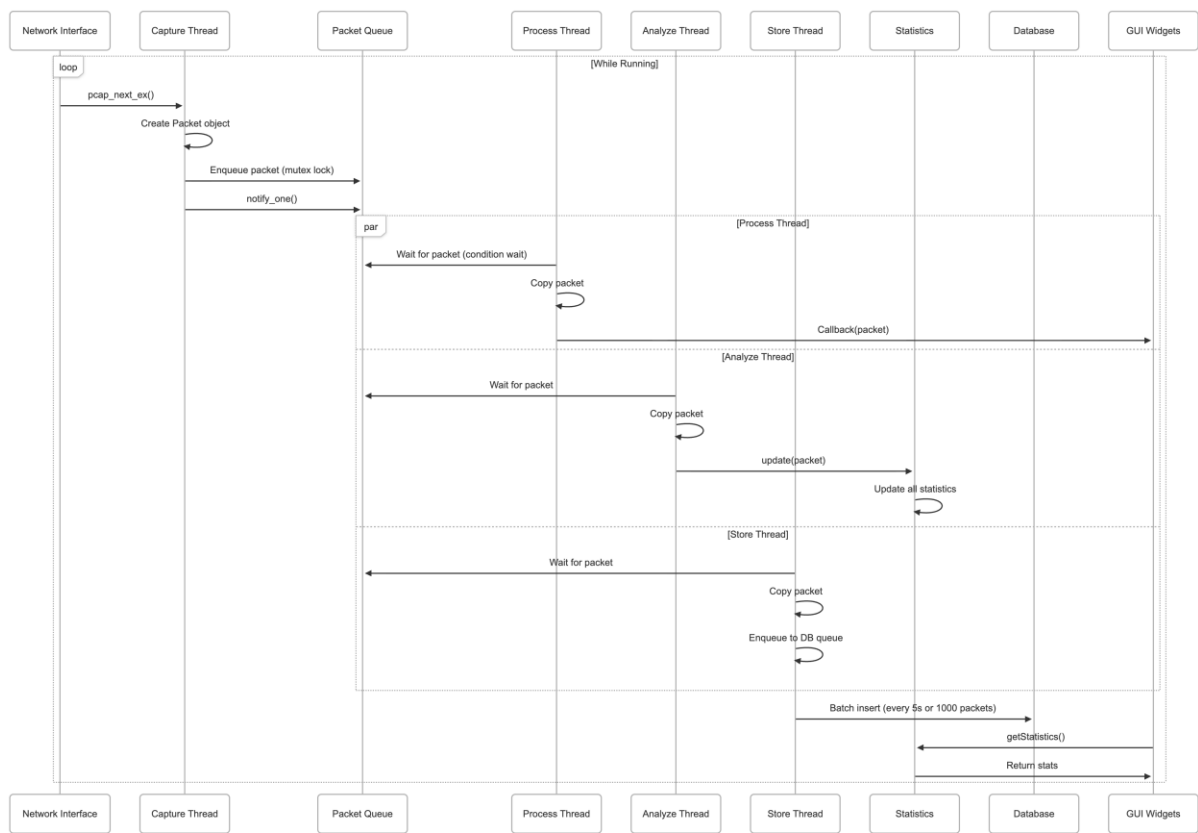
D -->|Events| H[CLI Interface]

E -->| Update | I[Statistics Object]
I -->| Read | G
I -->| Read | H

F -->| Batch Insert | J[SQLite Database]
J -->| Query | K[DataStore Queries]
K -->| Results | G
K -->| Results | H

style B fill:#e1f5ff
style C fill:#fff4e1
style I fill:#e8f5e9
style J fill:#f3e5f5

Packet Processing Flow



sequenceDiagram
participant NI as Network Interface

participant CT as Capture Thread
participant PQ as Packet Queue
participant PT as Process Thread
participant AT as Analyze Thread
participant ST as Store Thread
participant STAT as Statistics
participant DB as Database
participant GUI as GUI Widgets

loop While Running

NI->>CT: pcap_next_ex()
CT->>CT: Create Packet object
CT->>PQ: Enqueue packet (mutex lock)
CT->>PQ: notify_one()

par Process Thread

PT->>PQ: Wait for packet (condition wait)
PT->>PT: Copy packet
PT->>GUI: Callback(packet)

and Analyze Thread

AT->>PQ: Wait for packet
AT->>AT: Copy packet
AT->>STAT: update(packet)
STAT->>STAT: Update all statistics

and Store Thread

ST->>PQ: Wait for packet
ST->>ST: Copy packet
ST->>ST: Enqueue to DB queue

end

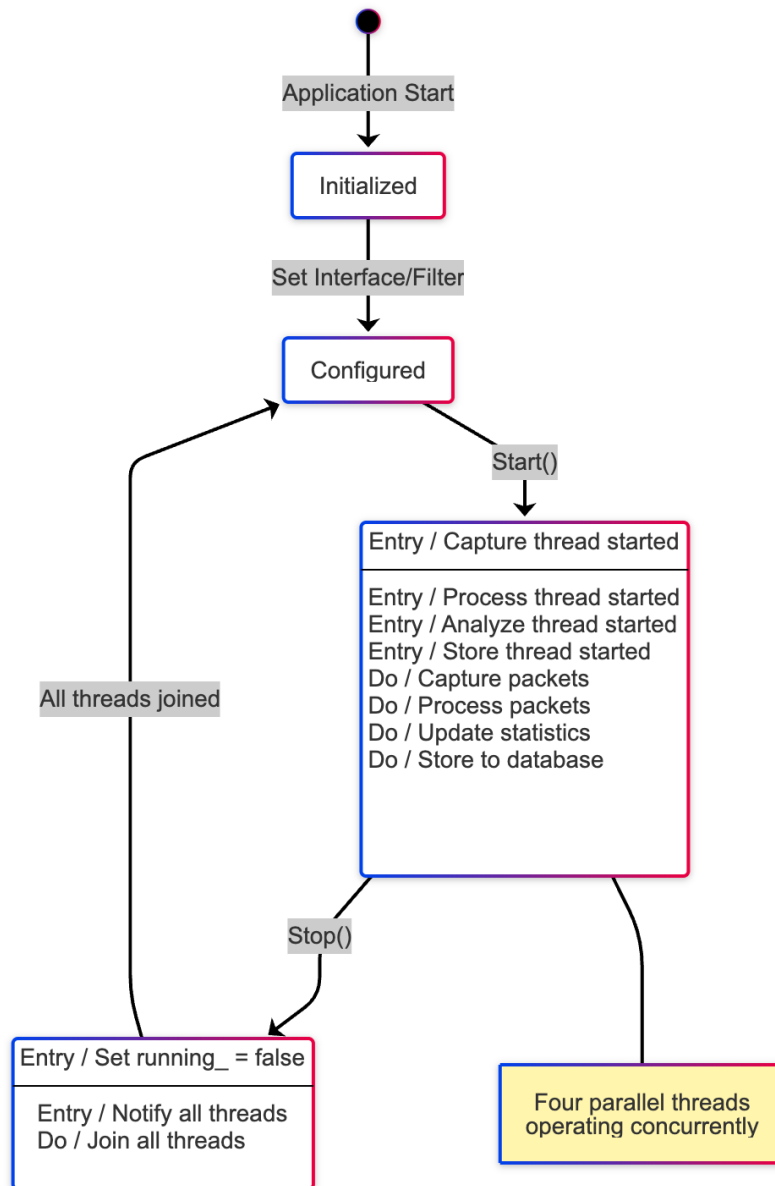
ST->>DB: Batch insert (every 5s or 1000 packets)

GUI->>STAT: getStatistics()

STAT->>GUI: Return stats

end

State Transition Diagram



stateDiagram-v2

[*] --> Initialized: Application Start

Initialized --> Configured: Set Interface/Filter

Configured --> Monitoring: Start()

Monitoring --> Stopping: Stop()

Stopping --> Configured: All threads joined

Monitoring: Entry / Capture thread started

Monitoring: Entry / Process thread started
Monitoring: Entry / Analyze thread started
Monitoring: Entry / Store thread started

Monitoring: Do / Capture packets
Monitoring: Do / Process packets
Monitoring: Do / Update statistics
Monitoring: Do / Store to database

Stopping: Entry / Set running_ = false
Stopping: Entry / Notify all threads
Stopping: Do / Join all threads

note right of Monitoring
 Four parallel threads
 operating concurrently
end note

Component Interactions

NetworkMonitor Component

Responsibilities:

- Packet capture coordination
- Thread lifecycle management
- Filter management
- Callback registration

Key Interactions:

- **With Statistics:** Provides statistics via `getStatistics()` (read-only)
- **With DataStore:** Passes packets to store thread via `storeThread()`
- **With GUI:** Accepts callbacks via `addPacketCallback()`

File: `include/core/NetworkMonitor.hpp:16-50`

Statistics Component

Responsibilities:

- Real-time statistical aggregation
- Protocol distribution tracking
- Host activity monitoring
- Connection state management
- Bandwidth calculation

Key Methods & Business Logic:

File: include/analysis/Statistics.hpp:36-94

Update Flow:

```
void Statistics::update(const Packet& packet) {  
    std::lock_guard<std::mutex> lock(mutex_);  
  
    total_packets_++;  
    total_bytes_ += packet.length;  
  
    updateProtocolStats(packet);  
    updateHostStats(packet);  
    updateConnectionStats(packet);  
    updateBandwidthStats(packet);  
    updateErrorStats(packet);  
  
    cleanupInactiveConnections();  
}
```

Protocol Statistics:

- Tracks packet count, byte count, error count per protocol
- Records first_seen and last_seen timestamps

Host Statistics:

- Tracks both source and destination hosts

- Maintains protocol breakdown per host
- Records activity timestamps

Connection Statistics:

- Generates canonical connection IDs
- Tracks packet count, bytes, retransmissions
- Marks connections as active/inactive
- Auto-cleanup after timeout

Bandwidth Statistics:

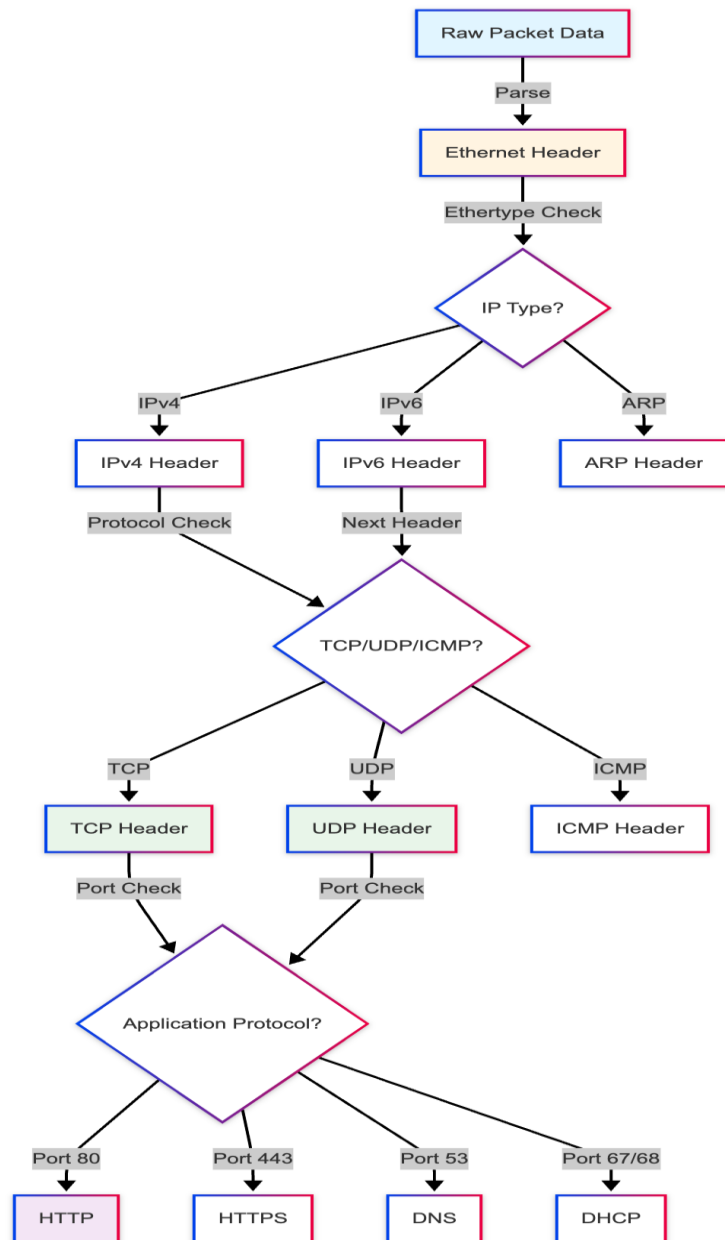
- Current bandwidth (bits/second)
- Average bandwidth over history
- History limited to 1 hour (3600 entries)

Packet Component

Responsibilities:

- Raw packet data storage
- Protocol parsing and identification
- Header field extraction
- Payload extraction

Parsing Hierarchy:



flowchart TD

A[Raw Packet Data] -->|Parse| B[Ethernet Header]

B -->|Etherertype Check| C{IP Type?}

C -->|IPv4| D[IPv4 Header]

C -->|IPv6| E[IPv6 Header]

C -->|ARP| F[ARP Header]

D -->|Protocol Check| G{TCP/UDP/ICMP?}

E -->|Next Header| G

G -->|TCP| H[TCP Header]

G -->|UDP| I[UDP Header]

G -->|ICMP| J[ICMP Header]

H -->|Port Check| K{Application Protocol?}

I -->|Port Check| K

K -->|Port 80| L[HTTP]

K -->|Port 443| M[HTTPS]

K -->|Port 53| N[DNS]

K -->|Port 67/68| O[DHCP]

G -->|TCP| H[TCP Header]
G -->|UDP| I[UDP Header]
G -->|ICMP| J[ICMP Header]

H -->|Port Check| K{Application Protocol?}
I -->|Port Check| K

K -->|Port 80| L[HTTP]
K -->|Port 443| M[HTTPS]
K -->|Port 53| N[DNS]
K -->|Port 67/68| O[DHCP]

style A fill:#e1f5ff
style B fill:#fff4e1
style H fill:#e8f5e9
style I fill:#e8f5e9
style L fill:#f3e5f5

File: src/protocols/Packet.cpp:12-262

Parsing Logic:

1. **Ethernet:** Check minimum header size → Extract ethertype
2. **IPv4/IPv6:** Extract source/dest addresses, TTL, check fragmentation
3. **TCP:** Extract ports, sequence/ack numbers, window size, payload offset
4. **UDP:** Extract ports, calculate payload offset
5. **Application:** Determine by port number (80→HTTP, 443→HTTPS, 53→DNS, 67/68→DHCP)

DataStore Component

Responsibilities:

- SQLite database management
- Packet persistence
- Batch insert optimization
- Query interface for historical data

Database Schema:

```
const char* sql = R"(
    CREATE TABLE IF NOT EXISTS packets (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        timestamp INTEGER NOT NULL,
        protocol TEXT NOT NULL,
        source_address TEXT NOT NULL,
        destination_address TEXT NOT NULL,
        source_port INTEGER,
        destination_port INTEGER,
        length INTEGER NOT NULL,
        is_fragmented BOOLEAN NOT NULL,
        is_malformed BOOLEAN NOT NULL,
        sequence_number INTEGER,
        acknowledgment_number INTEGER,
        window_size INTEGER,
        ttl INTEGER,
        tos INTEGER,
        payload BLOB
    );

    CREATE INDEX IF NOT EXISTS idx_packets_timestamp ON packets(timestamp);
    CREATE INDEX IF NOT EXISTS idx_packets_protocol ON packets(protocol);
    CREATE INDEX IF NOT EXISTS idx_packets_source ON packets(source_address);
    CREATE INDEX IF NOT EXISTS idx_packets_destination ON packets(destination_address);
);
```

Batch Insert Logic:

- Batch size: 1000 packets
- Flush interval: 5 seconds
- Uses SQLite transactions for atomicity
- Rollback on error

File: src/storage/DataStore.cpp:153-170

MainWindow Component (GUI)

Responsibilities:

- User interface coordination
- Menu and toolbar management
- Tab widget management
- Settings persistence
- Monitoring start/stop control

UI Update Logic:

- Timer-based updates (1 second interval)
- Updates only active tab
- Status bar shows packet count and bandwidth

File: src/gui/MainWindow.cpp:180-286

Widget Hierarchy:

- MainWindow (QMainWindow)
 - StatisticsWidget (Tab 0)
 - ConnectionsWidget (Tab 1)
 - PacketsWidget (Tab 2)
 - BandwidthWidget (Tab 3)

CommandLineInterface Component

Responsibilities:

- CLI command processing
- Interactive command loop
- Display formatting
- Statistics output

Commands:

- help - Show available commands
- stats - Display statistics
- connections - Show active connections

- packets - Display recent packets
- bandwidth - Show bandwidth history
- errors - Display error statistics
- filter <expr> - Set BPF filter
- clear - Clear filter
- save <file> - Save statistics
- export <file> - Export data
- quit/exit - Exit application

File: src/cli/CommandLineInterface.cpp:48-105

ConfigManager Component

Responsibilities:

- Configuration file parsing (INI format)
- Configuration value storage
- Thread-safe access
- Singleton pattern

Configuration Format:

[section]

key = value

Supported Types:

- String
- Integer
- Boolean (true/false)
- Double

File: src/config/ConfigManager.cpp:160-244

Logger Component

Responsibilities:

- Log message management

- Level filtering
- File and console output
- Timestamp formatting

Level Hierarchy:

- DEBUG (0)
- INFO (1)
- WARNING (2)
- ERROR (3)
- FATAL (4)

Output Rules:

- All messages → Log file
- ERROR/FATAL → Also to stderr

File: src/utils/Logger.cpp:51-97

Protocol Processing Logic

Ethernet Processing

File: src/protocols/Packet.cpp:33-59

Business Rules:

- Minimum packet size: sizeof(struct ether_header)
- Extract ethertype field (network byte order)
- Determine next layer: IPv4, IPv6, or ARP

IPv4 Processing

File: src/protocols/Packet.cpp:61-95

Business Rules:

- Extract source/destination IP addresses

- Extract TTL and TOS fields
- Detect fragmentation: (ip_off & IP_OFFMASK) != 0
- Determine transport protocol: TCP, UDP, or ICMP

IPv6 Processing

File: src/protocols/Packet.cpp:97-128

Business Rules:

- Extract source/destination IPv6 addresses
- Extract hop limit (equivalent to TTL)
- Determine next header: TCP, UDP, or ICMPv6

TCP Processing

File: src/protocols/Packet.cpp:130-153

Business Rules:

- Extract source/destination ports
- Extract sequence and acknowledgment numbers
- Extract window size
- Calculate payload offset: eth_header_size + ip_header_size + (tcp_header->th_off * 4)
- Extract payload if present

UDP Processing

File: src/protocols/Packet.cpp:155-175

Business Rules:

- Extract source/destination ports
- Calculate payload offset: eth_header_size + ip_header_size + udp_header_size
- Extract payload if present

Application Protocol Detection

File: src/protocols/Packet.cpp:193-208

Port-Based Rules:

- Port 80 → HTTP (TCP only)
- Port 443 → HTTPS (TCP only)
- Port 53 → DNS (UDP only)
- Port 67/68 → DHCP (UDP only)

Statistics & Analytics Business Logic

Protocol Distribution

Method: Statistics::getTopProtocols() **File:** src/analysis/Statistics.cpp:189-206

Logic:

1. Collect all protocol statistics
2. Sort by packet count (descending)
3. Return top N protocols

Host Distribution

Method: Statistics::getTopHosts() **File:** src/analysis/Statistics.cpp:208-225

Logic:

1. Aggregate packet counts per host
2. Sort by packet count (descending)
3. Return top N hosts

Connection Tracking

Method: Statistics::updateConnectionStats() **File:** src/analysis/Statistics.cpp:82-109

Logic:

1. Generate canonical connection ID
2. Track packet count, byte count
3. Detect retransmissions (TCP only)
4. Mark connection start/end times

5. Mark as active

Bandwidth Calculation

Method: Statistics::updateBandwidthStats() **File:** src/analysis/Statistics.cpp:111-137

Calculation:

- Per-packet: bandwidth += packet.length * 8 (bytes to bits)
- Per-second: Store current bandwidth, reset counter
- Average: Sum of history / count

Error Tracking

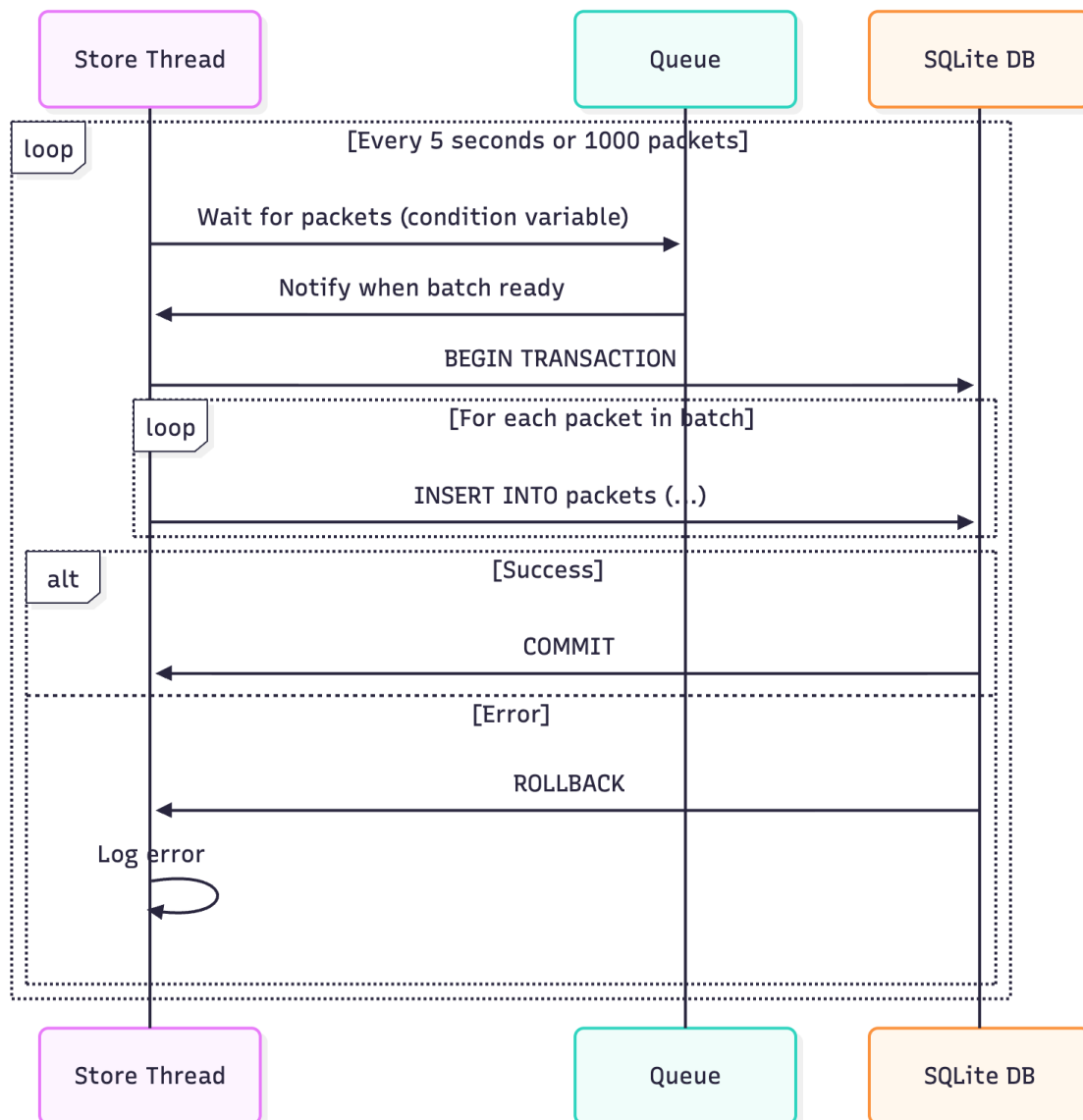
Method: Statistics::updateErrorStats() **File:** src/analysis/Statistics.cpp:139-143

Logic:

- Increment error count if packet.is_malformed == true
- Track errors per protocol
- Provide top errors query

Storage & Persistence Logic

Packet Storage Flow



sequenceDiagram

participant ST as Store Thread

participant Q as Queue

participant DB as SQLite DB

loop Every 5 seconds or 1000 packets

ST->>Q: Wait for packets (condition variable)

Q->>ST: Notify when batch ready

ST->>DB: BEGIN TRANSACTION

loop For each packet in batch

ST->>DB: INSERT INTO packets (...)

end

alt Success

DB->>ST: COMMIT

else Error

DB->>ST: ROLLBACK

ST->>ST: Log error

end

end

Batch Insert Rules

File: src/storage/DataStore.cpp:153-170

Triggers:

- Queue size $\geq$ 1000 packets (BATCH_SIZE)
- OR 5 seconds elapsed (FLUSH_INTERVAL)
- OR application shutdown

Transaction Management:

- Begin transaction before batch
- Commit on success
- Rollback on any error

Query Interface

Methods: getPacketsByProtocol(), getPacketsByHost(), getPacketsByTimeRange(), etc.

File: src/storage/DataStore.cpp:205-293

Query Pattern:

1. Prepare SQL statement with parameters
2. Bind parameters
3. Execute and fetch results
4. Convert rows to Packet objects (implementation incomplete in code)

User Interface Business Logic

GUI Start/Stop Logic

File: src/gui/MainWindow.cpp:266-286

Start Monitoring:

1. Call `monitor_->start()`
2. Set `is_monitoring_ = true`
3. Enable stop action, disable start action
4. Start update timer (1000ms interval)
5. Update status bar

Stop Monitoring:

1. Call `monitor_->stop()`
2. Set `is_monitoring_ = false`
3. Enable start action, disable stop action
4. Stop update timer
5. Update status bar

Filter Management

File: src/gui/MainWindow.cpp:208-224

Set Filter:

1. Show FilterDialog
2. Get filter expression
3. Call `monitor_->setFilter()`
4. If error → Show warning dialog

5. Update status bar

Clear Filter:

1. Call `monitor_->setFilter("")`
2. Update status bar

Settings Persistence

File: `src/gui/MainWindow.cpp:154-170`

Load Settings:

- Window geometry
- Window state
- Theme preference

Save Settings:

- Called on window close
- Saves geometry and state

Theme Management

File: `src/gui/MainWindow.cpp:288-310`

Dark Theme:

- Uses Fusion style
- Custom dark palette
- Applied on load if setting is "dark"

Light Theme:

- Uses Fusion style
- Standard palette

Error Handling & Edge Cases

Error Categories

1. Packet Capture Errors

Location: src/core/NetworkMonitor.cpp:101-103

Handling:

- Log error message
- Continue monitoring (don't stop)
- Error doesn't propagate

2. Filter Compilation Errors

Location: src/core/NetworkMonitor.cpp:33-40

Handling:

- Throw std::runtime_error
- Prevent monitoring start
- User must fix filter or clear it

3. Database Errors

Location: src/storage/DataStore.cpp:158-169

Handling:

- Rollback transaction on error
- Throw exception (may crash store thread)
- Log error before throw

4. Interface Open Errors

Location: src/core/NetworkMonitor.cpp:26-29

Handling:

- Throw `std::runtime_error` with pcap error message
- Prevent monitoring start

5. Malformed Packets

Location: Multiple in `src/protocols/Packet.cpp`

Handling:

- Set `is_malformed = true` flag
- Continue parsing as much as possible
- Track in error statistics

6. Logger Errors

Location: `src/utils/Logger.cpp:51-55`

Handling:

- If not initialized → output to `stderr`
- If initialized → write to log file

Edge Cases

Empty Packet Queue

Handling:

- Threads wait on condition variable
- Wake on: packet arrival OR shutdown signal
- Check `!running_ && queue.empty()` before exit

Concurrent Statistics Access

Handling:

- All statistics updates locked by single mutex
- Read operations also locked
- Ensures consistency

Connection Timeout

Handling:

- Cleanup runs after each packet update
- Removes inactive connections (>5 minutes)
- Prevents unbounded memory growth

Bandwidth History Overflow

Handling:

- Maximum 3600 entries (1 hour)
- Remove oldest entries when limit reached
- Maintains rolling window

Database Batch Overflow

Handling:

- Batch size limit: 1000 packets
- Time-based flush: 5 seconds
- Prevents queue from growing unbounded

Configuration Management

Configuration Loading

File: src/config/ConfigManager.cpp:160-192

Parse Rules:

- Section format: [section_name]
- Key-value format: key = value
- Ignore empty lines and comments (; or #)
- Trim whitespace

Type Detection:

1. Try boolean (true/false)
2. Try integer (stoi)
3. Try double (stod)
4. Default to string

Configuration Storage

Structure:

`std::unordered_map<Section, std::unordered_map<Key, Value>>`

Value Type:

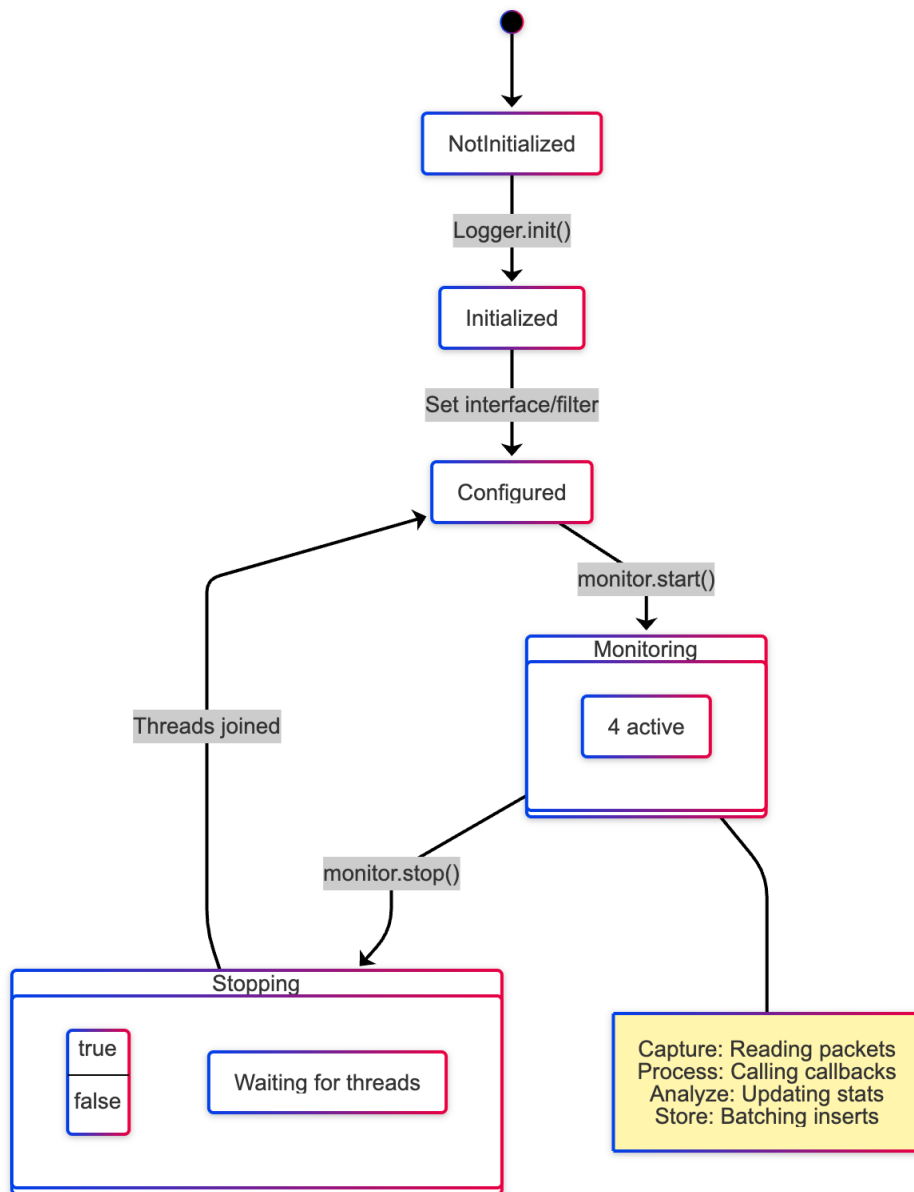
- `std::variant<std::string, int, bool, double>`

Thread Safety

- All operations protected by mutex
- Singleton pattern ensures single instance

State Management

Application States



stateDiagram-v2

[*] --> NotInitialized

NotInitialized --> Initialized: Logger.init()

Initialized --> Configured: Set interface/filter

Configured --> Monitoring: monitor.start()

Monitoring --> Stopping: monitor.stop()
Stopping --> Configured: Threads joined

Monitoring: State: running_ = true
Monitoring: State: 4 threads active

Stopping: State: running_ = false
Stopping: State: Waiting for threads

note right of Monitoring
 Capture: Reading packets
 Process: Calling callbacks
 Analyze: Updating stats
 Store: Batching inserts
end note

NetworkMonitor State

Variable: running_ (atomic bool)

State Transitions:

- false → true: On start()
- true → false: On stop()

Thread Coordination:

- All threads check running_ in loop condition
- Condition variable used for waiting
- notify_all() on stop to wake all threads

Statistics State

Collections:

- protocol_stats_: Map of protocol → ProtocolStats
- host_stats_: Map of host → HostStats
- connection_stats_: Map of connection_id → ConnectionStats

- `bandwidth_history_`: Vector of (timestamp, bandwidth) pairs

State Mutations:

- All mutations locked by `mutex_`
- Atomic counters for thread-safe increments
- Cleanup operations run periodically

DataStore State

Variable: `running_` (atomic bool)

Queue State:

- `packet_queue_`: Queue of packets to insert
- Protected by `queue_mutex_`
- Condition variable for thread signaling

Database State:

- SQLite connection maintained
- Transaction state (active/inactive)

Summary

This Network Monitoring System implements a sophisticated multi-threaded packet capture and analysis pipeline with the following key business logic components:

1. **Multi-threaded Architecture:** Four parallel threads handle capture, processing, analysis, and storage
2. **Protocol Hierarchy:** Layered parsing from Ethernet → IP → Transport → Application
3. **Real-time Statistics:** Atomic updates across protocol, host, connection, and bandwidth dimensions
4. **Batch Storage:** Optimized database inserts using transactions and batching
5. **Dual Interface:** GUI and CLI interfaces with shared backend logic
6. **Error Resilience:** Graceful error handling with logging and state recovery

The system follows modern C++ practices with RAII, smart pointers, and thread-safe synchronization primitives, making it suitable for production network monitoring applications.

Appendix: File Reference Map

Component	Header File	Implementation File
Core Monitor	include/core/NetworkMonitor.hpp	src/core/NetworkMonitor.cpp
Packet Parser	include/protocols/Packet.hpp	src/protocols/Packet.cpp
Statistics	include/analysis/Statistics.hpp	src/analysis/Statistics.cpp
Data Store	include/storage/DataStore.hpp	src/storage/DataStore.cpp
Main Window	include/gui/MainWindow.hpp	src/gui/MainWindow.cpp
CLI	include/cli/CommandLineInterface.hpp	src/cli/CommandLineInterface.cpp
Config	include/config/ConfigManager.hpp	src/config/ConfigManager.cpp
Logger	include/utils/Logger.hpp	src/utils/Logger.cpp
Main Entry	N/A	src/main.cpp

Document End